Understanding Airbnb Pricing in New York City: A Data-Driven Analysis

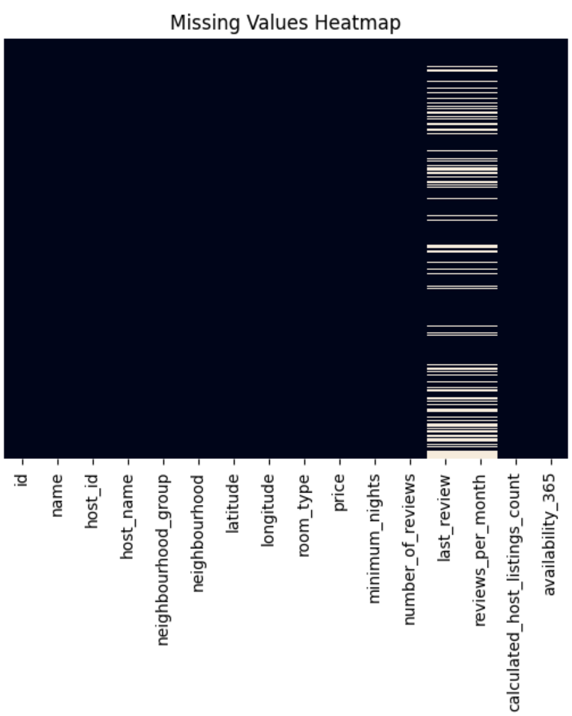
By Sejal Mogalgiddi,
Apoorva Thirukazhukundram Shakila Raja,
Mahi Shah,
Ashritha Koreboina

Introduction

New York City hosts one of the busiest Airbnb markets. There is high variability in Airbnb prices due to differences in location, property types, and demand. An added benefit of understanding the market is also important for guests, as price influences both affordability and value. For these reasons,s it is beneficial to analyze the market and allow hosts and guests to make more informed decisions and have the best experience possible.

For our dataset, we took New York City Airbnb Open Data (2019), which contains roughly 48,895 individual listings spanning until 5 boroughs. Individual data points include price, neighborhood group, room type, number of reviews, availability, minimum nights, host name, host since, and more. This dataset features numerical and categorical variables, so we had to do some preprocessing before creating our models.

In this project, we will do our EDA and create machine learning models that allow us to better understand how the different features of a listing can affect Airbnb prices. We will look for patterns in the data and prepare the data for modeling by cleaning missing data and outliers. We will create multiple models and determine which model is the best fit.



This project can be useful because it allows us to take what we learn in class and see how it can be applied to a real-world marketplace. By learning what the important factors and patterns are that affect Airbnb prices, people who own Airbnb's can make more educated decisions when pricing their listings. It also allows people to understand how pricing works in the short-term rental market, and for guests to see if they are really getting the best deal.

Missing Values Analysis

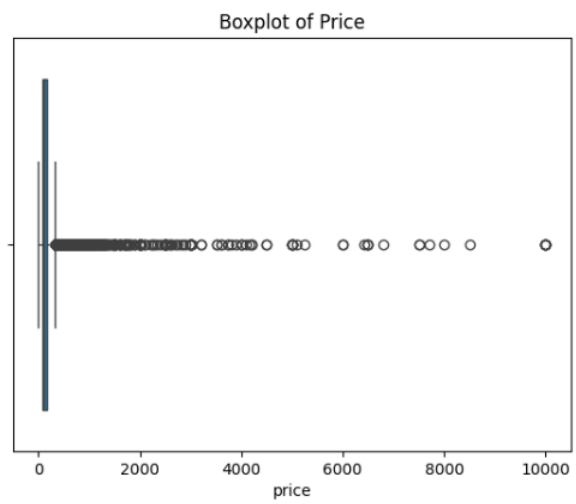
We first analyzed missing values in the dataset to understand data quality and identify any preprocessing needs.

From the results:

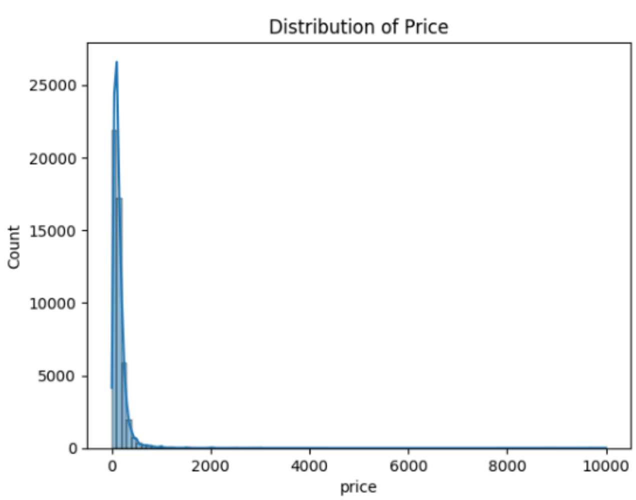
- Most columns had no missing values
- However, the following columns had missing data:
  - last_review, reviews_per_month: 10,052 missing values
  - Minor missing values in:

- name (19)
- host_name (21)

Summary Statistics



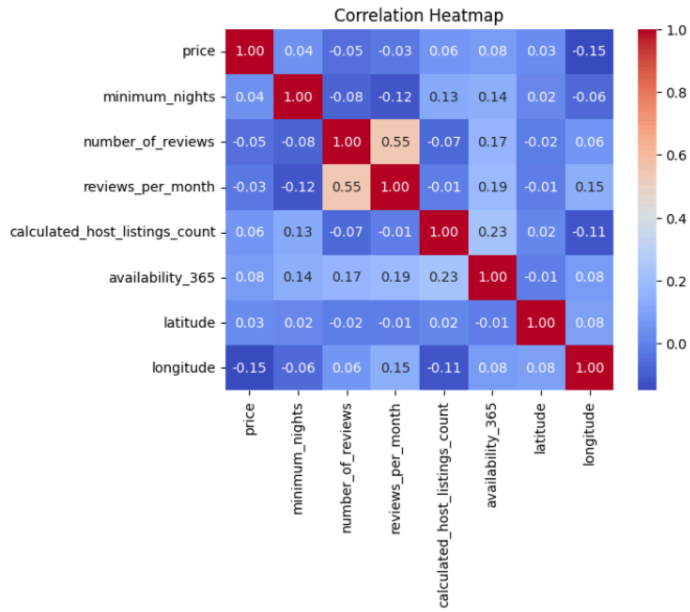
We looked at the summary stats for the dataset, just to get a sense of how the numbers are spread out and what kind of ranges they have. The distribution of price stands out right away, with a mean of around 152 but a median of 106. We can see from these summary statistics that the distribution is skewed to the right. We also see many outliers with our highest value reaching 10,000. This extremeness can cause all of our results to become extremely skewed, which may cause future problems. To solve this issue, we would need to use a transformation on the price



For minimum_nights, the median is 3, but it goes all the way to 125; this shows that the pricing on some listings may not be realistic and that we would need to take a closer look at these listings. The number_of_reviews varies from zero up to 629; this entails that some listings have many reviews while some have no reviews at all. Availability_365 ranges from 0 to 365 days; this means some listings can be booked for the whole year, and others are hardly ever booked. This wide range in the data showcases variability which provides many insights such as how popular a listing may be, which listings may need updates, etc.

Overall, we can see that some extreme outliers need to be observed; we see that our data ranges widely and provides valuable insights once we take a closer look. In data preprocessing, we can fix our issue of outliers and transform the price so that our data is no longer skewed by disproportionate prices on certain listings.

Correlation Analysis of Numerical Features

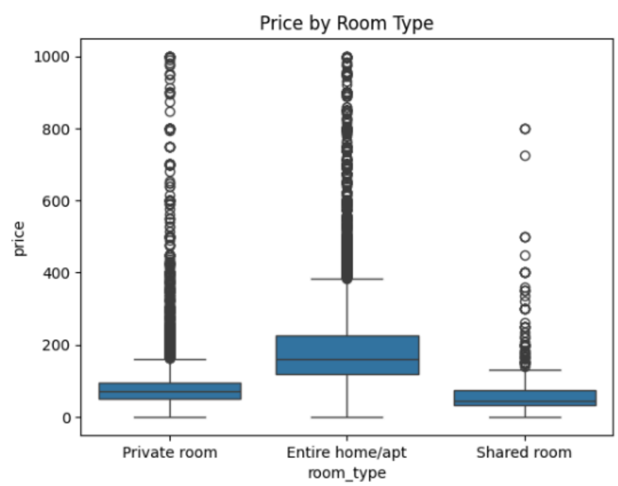


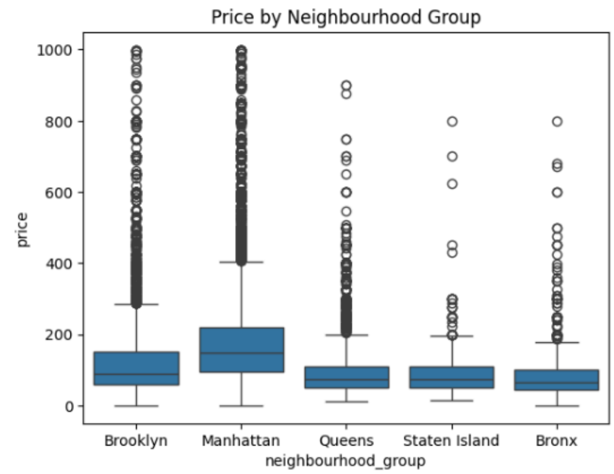
We analyzed the correlation between numerical features using a heatmap to understand the relationships between variables. From the results, we observe that the correlation between **price** and other numerical features is generally weak, with values close to zero. This indicates that there is **no strong linear relationship** between price and any single numerical variable. Therefore, price is likely influenced by **more complex, nonlinear interactions** between multiple features rather than simple linear patterns.

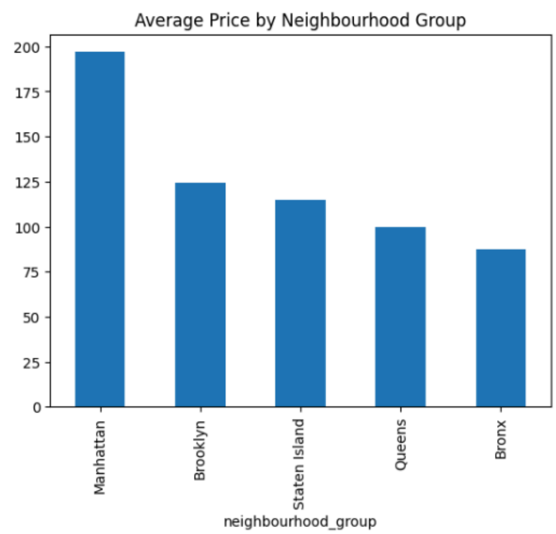
Additionally, we observe a relatively strong positive correlation between **number_of_reviews** and **reviews_per_month** (around 0.55), which suggests that listings with more total reviews tend to receive reviews more frequently. This indicates higher listing activity. However, this also introduces **multicollinearity**, which can negatively affect linear models by making coefficients unstable.

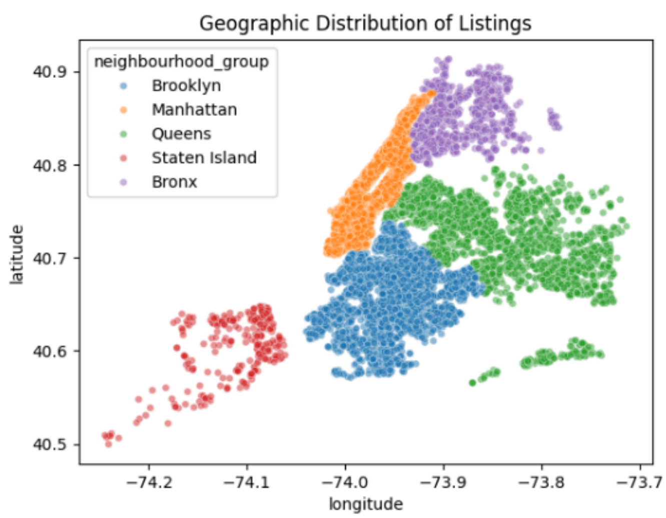
Overall, these findings suggest that **tree-based models (such as Random Forest and Gradient Boosting)** are more suitable for this dataset, as they can better capture nonlinear relationships and are less affected by multicollinearity compared to linear models. Tree-based models are also more accurate for prediction when the dataset exhibits skewed distributions and outliers.

Exploratory Data Analysis:

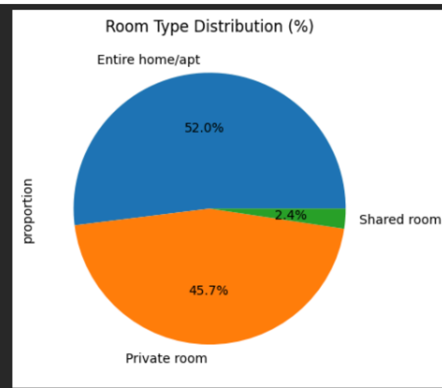








Room type and location really affect how much an Airbnb costs. Entire homes or apartments tend to be more expensive, but can vary. On the other hand, private rooms and shared rooms are cheaper. Prices also depend on the borough; for example, Manhattan is the most expensive, followed by Brooklyn, while Queens and the Bronx are more affordable. There are also more listings in Manhattan and Brooklyn, which shows that these areas are the busiest for Airbnb.




```
corr_target = df[numeric_cols].corr()["price"].sort_values(ascending=False)
print(corr_target)

price                             1.000000
availability_365                  0.081829
calculated_host_listings_count    0.057472
minimum_nights                    0.042799
latitude                          0.033939
reviews_per_month                -0.030608
number_of_reviews                -0.047954
longitude                        -0.150019
Name: price, dtype: float64
```

In the analysis, it seems that price is not strongly affected by any of the numerical features, as all the correlations we saw are close to 0, so no feature has a significant influence. The availability and the host listing count have only very small positive relationships with price, while longitude has a weak negative relationship. This is also confirmed in the scatter plot of price vs number of reviews, in which no obvious linear pattern appears. Instead, we see that listings with high reviews tend to cluster at lower prices. This suggests that more affordable listings get booked more often, and guests are more likely to write reviews for those listings. Expensive listings get fewer reviews because they don't get booked that often.

From the categorical and geographic analysis, **room type and location are both crucial in determining the price of a listing**. Entire homes/apartments make up the majority of listings (~52%) and are significantly more expensive, while private rooms (~46%) and shared rooms (~2.4%) are cheaper. Additionally, borough-level differences show that **Manhattan listings have the highest prices**, followed by Brooklyn, while Queens and the Bronx are more affordable. The geographic distribution further supports that listings are focused on high-demand areas like Manhattan and Brooklyn since these locations attract more tourists.

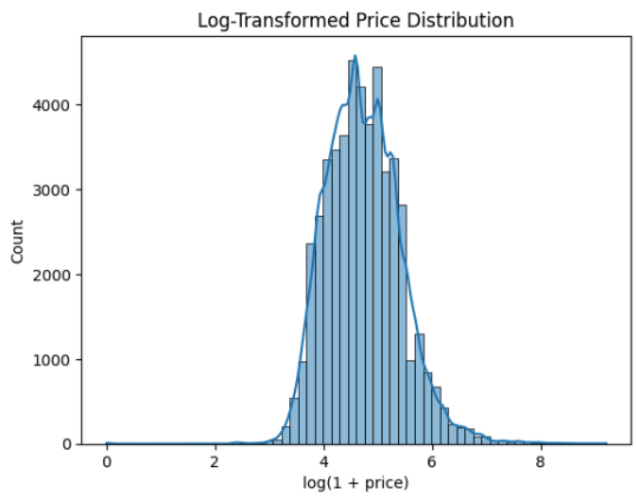
Business Insights

- Hosts can **increase bookings by pricing competitively**, as lower-priced listings tend to receive more reviews and higher activity.
- Investing in **entire homes/apartments in prime locations (e.g., Manhattan)** can yield higher revenue, though demand may be lower
- Location is a key driver of price, so hosts should consider **neighborhood demand when setting prices.**

- Since price depends on multiple interacting factors (not just one feature), **dynamic pricing strategies** are more effective than fixed pricing

Overall, price is driven more by **categorical and geographic factors rather than simple numerical relationships**, indicating the importance of capturing complex patterns in modeling.

Log Transformation of Price



A log transformation was applied to the price variable to reduce skewness and improve its distribution. As shown in the plot, the transformation helps reduce the extreme skew in price and makes the target variable easier to model, so we can evaluate it better. This also helps with extreme outliers and improves model performance.

Data Preprocessing

In this step, we cleaned the dataset and got it ready for modeling. First, we created a new target variable by taking the log of the price, and we removed very high prices (above 1000) to make the data less skewed. For missing values in review-related columns, we filled them in using appropriate methods. We also created a new feature called **days_since_last_review** using the date information.

Next, we removed unnecessary ID columns and split the data into 80% training and 20% testing. Numerical features were scaled, and missing values were filled using the median, while categorical features were converted into numbers using one-hot encoding. After all these steps, the number of features increased from 11 to 234, which helps the models learn more detailed patterns.

Models Preamble

Here we want to compare different models and see which one works best for the data. We used a combination of models, i.e., Linear Regression, Decision Tree, Random Forest, and Gradient Boosting. This was so that we could compare simple methods with more advanced ones. We selected these models based on the findings we made during the data analysis, where it seemed that the relationship between features and price is not entirely linear; more complex models might perform better.

Model Evaluation and Results

To evaluate model performance, we used metrics such as Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2 score. These metrics allow us to measure prediction accuracy and compare how well each model generalizes to unseen data.

	RMSE	MAE	R2
Random Forest	0.411732	0.298569	0.621276
Gradient Boosting	0.420845	0.308920	0.604326
Linear Regression	0.443591	0.327849	0.560398
Decision Tree	0.566092	0.410944	0.284075

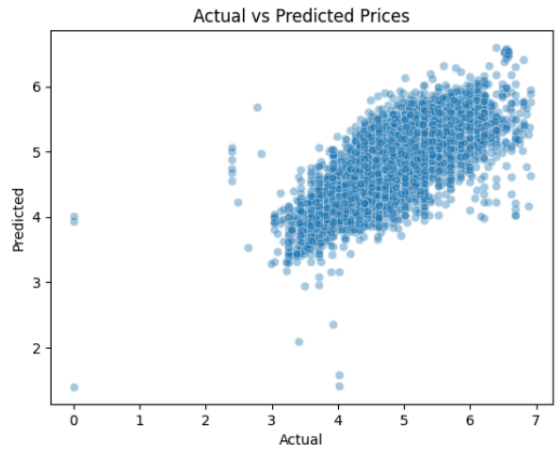
```
Model Comparison:
                       RMSE       MAE        R2
Linear Regression  0.443591  0.327849  0.560398
Decision Tree      0.566092  0.410944  0.284075
Random Forest      0.411732  0.298569  0.621276
Gradient Boosting  0.420845  0.308920  0.604326
```

- **Random Forest** performed the best with RMSE ≈ 0.412 and $R^2 \approx$ 0.62
- **Gradient Boosting** performed similarly, with slightly higher error and slightly lower R^2 compared to Random Forest
- **Linear Regression** showed moderate performance
- **The decision tree** performed the worst due to overfitting

Overall, the results confirm that tree-based models are more effective for this dataset as they better capture nonlinear relationships and handle high-dimensional data.

Feature Importance and Model Performance Analysis

	Feature	Importance
231	cat__room_type_Entire home/apt	0.403152
1	num__longitude	0.135087
0	num__latitude	0.122736
6	num__availability_365	0.063263
7	num__days_since_last_review	0.044309
2	num__minimum_nights	0.043152
4	num__reviews_per_month	0.041453
3	num__number_of_reviews	0.034469
10	cat__neighbourhood_group_Manhattan	0.028218
5	num__calculated_host_listings_count	0.026758
140	cat__neighbourhood_Midtown	0.006479
232	cat__room_type_Private room	0.005278
233	cat__room_type_Shared room	0.003519
26	cat__neighbourhood_Bedford-Stuyvesant	0.002222
225	cat__neighbourhood_Williamsburg	0.001954



To understand how the model makes predictions, we looked at feature importance from the best model, which is Random Forest. The results show that **room type**, especially entire home/apartment, has the biggest impact on price compared to all other features. This matches what we saw earlier—entire homes are usually much more expensive than private or shared rooms.

Location is also very important. Features like latitude and longitude are among the top factors, which means where the listing is located strongly affects the price. Other features like availability (availability_365), days since last review, and minimum nights also have some influence, showing that booking conditions and activity matter too.

On the other hand, features like the number of reviews, reviews per month, and specific neighborhood names (like Manhattan or Midtown) have less impact. This shows that price is not controlled by just one factor, but by a mix of different features.

To check how well the model works, we compared actual log-transformed prices with predicted log-transformed prices using a scatter plot. Since our target variable is log_price, the model is predicting values in log scale and not raw dollars. The plot shows a clear upward trend, meaning the model is able to capture the general pattern pretty well. Most points are close to a diagonal line, so predictions are usually close to the real values.

However, there is still some spread in the points, especially for higher prices. This means the model struggles a bit with very expensive listings, which makes sense because those prices vary a lot.

Overall, the model does a good job of capturing general pricing trends, and it shows that room type and location are the most important factors affecting Airbnb prices.

Limitations

During our model predictions, we came across a few limitations. The major limitation we found was that the model predicts in log-transformed prices rather than actual dollars. This means we cannot take the direct predictions as the actual predictions; we must scale them back to normal dollars exponentially. Additionally, our dataset doesn't take other external factors like seasonality, demand, and events into account, which may influence Airbnb prices drastically. The last limitation we saw was that our model struggled with very high price levels due to being trained on moderate price levels. Luxury Airbnb listings tend to have more variability, which has many factors; this limits the model's ability to fully capture real-world pricing behavior.

Conclusion

In this project, we analyzed the New York City Airbnb dataset to understand what affects listing prices and to build prediction models. We found that prices vary a lot and are not determined by just one factor, but by a combination of features.

Room type and location turned out to be the most important. Entire homes and listings in places like Manhattan are usually more expensive, while private and shared rooms cost less. We also saw that cheaper listings often get more reviews, which may mean they are booked more often.

Among the models, tree-based methods performed the best, with Random Forest giving the strongest results. This shows that pricing is complex and not purely linear. Overall, the project shows that Airbnb prices are influenced by multiple factors, and understanding these can help hosts set better prices.

Appendix: Key Code Snippets

In this section, key implementation steps are shown to highlight major coding aspects such as preprocessing, model selection, and the evaluation process.

Code Snippet 1: Preprocessing

```
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))])

#combine the results
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_cols),
        ("cat", categorical_transformer, categorical_cols)])
```

Code Snippet 2: Models used

```
models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=10),
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=10),
    "Gradient Boosting": GradientBoostingRegressor(random_state=10)
}
```

Code Snippet 3: Evaluation loop

```
for name, model in models.items():
    # Train model
    model.fit(X_train_processed, y_train)

    # Predict
    y_pred = model.predict(X_test_processed)

    # Metrics
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    # Store results
    results[name] = {
        "RMSE": rmse,
        "MAE": mae,
        "R2": r2
    }
```